

REST APIs - Master Notes

A Complete Reference for Interview Preparation, Production Work, Real-World Projects, and System Understanding.

Java Full Stack Master Course

Table of Contents

REST APIS — MASTER NOTES	5
A Complete Reference Book for Interviews, Production Work & System Understanding	5
TOPIC 1: REST PRINCIPLES	5
1. Introduction	5
1.1 What is it?	5
1.2 Why was it introduced	5
2. The Six REST Architectural Constraints	5
3. The "Uniform Interface" — Four Sub-Principles	6
4. Real Life Analogy	6
5. "RESTful" vs Truly "REST-compliant" — A Real-World Nuance	6
6. Frequently Asked Interview Questions	6
TOPIC 2: HTTP METHODS	7
1. Introduction	7
2. The Core Methods — Full Reference Table	7
3. "Safe" vs "Idempotent" — A Frequently Confused Pair	7
4. PUT vs PATCH — A Critical, Frequently-Tested Distinction	8
5. Real Life Analogy	8
6. Edge Cases	8
7. Common Mistakes	8
8. Frequently Asked Interview Questions	9
TOPIC 3: STATUS CODES	9
1. Introduction	9
2. The Five Categories	9
3. The Status Codes You'll Actually Use — Full Reference	10
4. 401 vs 403 — A Frequently Confused, Frequently-Tested Pair	10
5. Real Life Analogy	10
6. Edge Cases	11

7. Common Mistakes	11
8. Frequently Asked Interview Questions	11

TOPIC 4: REQUEST & RESPONSE **11**

1. Introduction	11
2. Anatomy of an HTTP Request	12
3. Anatomy of an HTTP Response	12
4. Key Headers to Know	12
5. Frequently Asked Interview Questions	12

TOPIC 5: DTOS (DATA TRANSFER OBJECTS) **12**

1. Introduction	13
2. Why NOT Return @Entity Objects Directly (Recap + Deeper Reasoning)	13
3. Code Example — Entity-to-DTO Mapping	13
4. Mapping Libraries (For Larger Projects)	14
5. Real Life Analogy	14
6. Common Mistakes	14
7. Frequently Asked Interview Questions	14

TOPIC 6: VALIDATION (CROSS-REFERENCE) **14**

TOPIC 7: VERSIONING **15**

1. Introduction	15
2. The Four Common Versioning Strategies	15
Strategy 1: URI Path Versioning (Most Common, Simplest)	15
Strategy 2: Query Parameter Versioning	15
Strategy 3: Custom Header Versioning	15
Strategy 4: Media Type (Content Negotiation) Versioning	15
3. Comparison Table	15
4. Real Life Analogy	15
5. Edge Cases	16
6. Best Practices	16
7. Frequently Asked Interview Questions	16

TOPIC 8: ERROR HANDLING (CROSS-REFERENCE)	16
--	-----------

TOPIC 9: HATEOAS	16
-------------------------	-----------

1. Introduction	17
2. Without HATEOAS (Typical, Most Common Real-World APIs)	17
3. With HATEOAS	17
4. Spring HATEOAS — Code Example	17
5. Real Life Analogy	18
6. Why Most Real-World APIs SKIP HATEOAS	18
7. Frequently Asked Interview Questions	18

TOPIC 10: OPENAPI/SWAGGER (CROSS-REFERENCE)	18
--	-----------

Revision Notes (Entire REST APIs Section Consolidated)	19
Homework (Covers Entire REST APIs Section)	19
FINAL SUMMARY — REST APIS COMPLETE	19

REST APIS — MASTER NOTES

A Complete Reference Book for Interviews, Production Work & System Understanding

TOPIC 1: REST PRINCIPLES

1. Introduction

1.1 What is it?

REST (Representational State Transfer) is an ARCHITECTURAL STYLE for designing networked applications, defined by Roy Fielding in his 2000 doctoral dissertation. It's a set of CONSTRAINTS/PRINCIPLES — NOT a protocol or a library — that, when followed, produce APIs that are scalable, simple, and consistent. Everything you've built in the Spring Boot section's `@RestController` examples has been an IMPLEMENTATION of these principles, using HTTP as the transport.

1.2 Why was it introduced

Before REST became dominant, web service APIs commonly used SOAP (a rigid, XML-heavy, verbose protocol with strict contracts). REST emerged as a SIMPLER alternative, leveraging HTTP's EXISTING, well-understood semantics (methods, status codes, headers) rather than inventing an entirely new protocol layer on top — resulting in APIs that are easier to build, test (even just via a browser or `curl`), cache, and scale.

2. The Six REST Architectural Constraints

Constraint	Meaning
Client-Server	UI concerns (client) are separated from data storage concerns (server) — they can evolve independently
Statelessness	Each request from client to server must contain ALL information needed to understand it — the server stores NO client session state between requests (directly connects to why JWT, from Spring Security, is a natural fit for REST APIs, unlike traditional <code>HttpSession</code> -based auth)
Cacheability	Responses must explicitly define themselves as cacheable or not, allowing clients/intermediaries to reuse data and improve performance
Uniform Interface	A consistent, standardized way of interacting with resources (this is the DEFINING characteristic of REST — covered in detail below)
Layered System	A client cannot necessarily tell whether it's connected directly to the end server or an intermediary (load balancer, cache, gateway) — layers can be added transparently
Code on Demand (optional)	Servers can optionally extend client functionality by transferring executable code (e.g., JavaScript) — rarely relevant for typical backend APIs

3. The "Uniform Interface" — Four Sub-Principles

- 1 **Resource identification:** every piece of data is a RESOURCE, identified by a URI (`/api/employees/42`).
- 2 **Manipulation through representations:** the client interacts with a REPRESENTATION of a resource (typically JSON), not the resource itself directly.
- 3 **Self-descriptive messages:** each message includes enough information to describe how to process it (e.g., `Content-Type: application/json`).
- 4 **HATEOAS** (Hypermedia As The Engine Of Application State) — covered in its own dedicated topic later; the idea that responses include LINKS to related actions/resources.

4. Real Life Analogy

- **Restaurant:** REST is like a restaurant chain adopting a STANDARDIZED way of taking orders across EVERY location — every branch uses the SAME menu format, the SAME ordering process (uniform interface), each order ticket contains EVERYTHING the kitchen needs with no reliance on "remembering" previous unrelated orders (statelessness), and a customer doesn't need to know if they're talking to a franchise location or the flagship restaurant (layered system) — it all works the same way regardless.

5. "RESTful" vs Truly "REST-compliant" — A Real-World Nuance

Most APIs called "RESTful" in industry (including the vast majority you'll build in Spring Boot) **do NOT strictly implement HATEOAS** or all six constraints perfectly — the term "RESTful" has come to colloquially mean "an HTTP JSON API following REST-INSPIRED conventions" (proper use of HTTP methods/status codes, resource-based URLs) rather than a textbook-pure implementation of Fielding's full architectural style. This is a genuinely useful, frequently-tested interview distinction.

6. Frequently Asked Interview Questions

- 1 **Who defined REST, and in what context?** Roy Fielding, in his 2000 doctoral dissertation.
- 2 **Is REST a protocol?** No — it's an architectural STYLE/set of constraints, typically implemented using HTTP as the underlying protocol.
- 3 **What does "statelessness" mean in REST, and how does it connect to authentication?** Each request must contain all information needed to process it, with no server-side session state between requests — this is exactly why JWT-based (stateless) authentication (Spring Security section) is a more natural REST fit than traditional server-side `HttpSession`-based authentication.
- 4 **What is the "Uniform Interface" constraint?** REST's defining characteristic — a consistent way of identifying resources (URIs), manipulating them via representations, self-descriptive messages, and (ideally) HATEOAS.
- 5 **Do most real-world "RESTful" APIs fully satisfy all of Fielding's constraints?** No — most implement REST-INSPIRED conventions (proper HTTP method/status code usage, resource-based URLs) without full HATEOAS compliance; "RESTful" is used more loosely in industry practice.

TOPIC 2: HTTP METHODS

1. Introduction

What is it? HTTP methods (verbs) indicate the INTENDED ACTION to perform on a resource. Choosing the CORRECT method for each operation is central to building a properly RESTful API.

2. The Core Methods — Full Reference Table

Method	Purpose	Idempotent?	Safe?	Has a Request Body?
GET	Retrieve a resource	Yes	Yes	No
POST	Create a NEW resource (or trigger a non-idempotent action)	No	No	Yes
PUT	REPLACE an entire existing resource	Yes	No	Yes
PATCH	PARTIALLY update an existing resource	No (typically)	No	Yes
DELETE	Remove a resource	Yes	No	Usually no

3. "Safe" vs "Idempotent" — A Frequently Confused Pair

- **Safe:** the operation does NOT change server state at all (read-only) — only GET (and HEAD) are safe.
- **Idempotent:** making the SAME request MULTIPLE times produces the SAME end result as making it ONCE (though the response itself might differ, e.g., a second DELETE might return 404 instead of 204, but the RESOURCE'S final state is identical either way). GET, PUT, DELETE are idempotent; POST is generally NOT (calling it twice typically creates TWO resources).

4. PUT vs PATCH — A Critical, Frequently-Tested Distinction

```
// PUT - REPLACES the ENTIRE resource. The client must send ALL fields, even unchanged ones.
@PutMapping("/{id}")
public Employee replaceEmployee(@PathVariable Long id, @RequestBody Employee fullEmployee) {
    fullEmployee.setId(id);
    return employeeRepository.save(fullEmployee); // ANY field NOT in the request body gets
    OVERWRITTEN/nulled!
}

// PATCH - PARTIALLY updates ONLY the fields actually provided
@PatchMapping("/{id}")
public Employee updateEmployeePartial(@PathVariable Long id, @RequestBody Map<String, Object> updates)
{
    Employee emp = employeeRepository.findById(id).orElseThrow();
    updates.forEach((key, value) -> {
        switch (key) {
            case "salary" -> emp.setSalary((Double) value);
            case "email" -> emp.setEmail((String) value);
        }
    });
    // only the fields ACTUALLY present in 'updates' get touched
    return employeeRepository.save(emp);
}
```

5. Real Life Analogy

- **Bank:** PUT is like submitting a COMPLETELY NEW, full account application form to REPLACE your existing one entirely — any section you leave blank is treated as intentionally CLEARED. PATCH is like submitting a small "change of address" SLIP that updates ONLY that one specific field, leaving everything else on your account record untouched.

6. Edge Cases

Case	Result
Sending a PUT request with only SOME fields (missing others)	Missing fields are typically OVERWRITTEN to null/default — a common, serious real-world bug when a client mistakenly uses PUT intending PATCH semantics
Calling DELETE on an already-deleted resource	Often returns 404 Not Found on the SECOND call — the RESOURCE's final state (deleted) is idempotent, even though the specific RESPONSE code can differ between calls
Calling POST twice with identical data (e.g., due to a network retry)	Can create TWO duplicate resources — a genuine real-world problem, often mitigated with client-generated idempotency keys

7. Common Mistakes

- Using PUT when PATCH semantics were actually intended, accidentally nulling out fields not included in the request.
- Using GET for operations that modify state (a serious violation of the "safe" principle — also breaks caching/prefetching assumptions, since browsers/crawlers may issue GET requests speculatively).

- Not handling `POST` retry-induced duplicate creation in operations where this genuinely matters (e.g., payments).

8. Frequently Asked Interview Questions

- 1 **What's the difference between "safe" and "idempotent"?** Safe means the operation doesn't change server state at all; idempotent means repeating the SAME request produces the same end resource state, even if it DOES change state (like `DELETE`).
- 2 **Is `POST` idempotent?** No — generally, calling it multiple times creates multiple distinct resources.
- 3 **What's the key difference between `PUT` and `PATCH`?** `PUT` replaces the ENTIRE resource (missing fields are typically cleared); `PATCH` updates ONLY the specific fields provided.
- 4 **Why is it a serious mistake to use `GET` for an action that modifies data?** It violates the "safe" method principle, and can cause UNINTENDED side effects from browser prefetching, crawlers, or caching layers that assume `GET` requests are read-only.

TOPIC 3: STATUS CODES

1. Introduction

What is it? HTTP status codes are 3-digit numbers indicating the RESULT of a request, grouped into 5 categories by their first digit — choosing the CORRECT status code is essential for building a self-descriptive, properly-behaving REST API.

2. The Five Categories

Range	Category	Meaning
1xx	Informational	Request received, processing continues (rarely used directly in typical REST APIs)
2xx	Success	The request was successfully processed
3xx	Redirection	Further action needed to complete the request
4xx	Client Error	The CLIENT made a mistake (bad request, unauthorized, not found)
5xx	Server Error	The SERVER failed to process an otherwise valid request

3. The Status Codes You'll Actually Use — Full Reference

Code	Name	When to Use
200 OK	Success	Standard success for GET/PUT/PATCH
201 Created	Resource created	Successful POST that creates a NEW resource
204 No Content	Success, no body	Successful DELETE, or an update with nothing meaningful to return
400 Bad Request	Invalid input	Malformed JSON, failed validation (@Valid, from Spring Boot section)
401 Unauthorized	Not authenticated	Missing/invalid credentials — the user's IDENTITY isn't established
403 Forbidden	Not authorized	Identity IS known, but the user lacks PERMISSION for this action
404 Not Found	Resource doesn't exist	The requested URI/resource doesn't exist
409 Conflict	State conflict	E.g., trying to create a resource that already exists (duplicate email)
422 Unprocessable Entity	Semantically invalid	The request is well-formed but fails business validation rules (some APIs use this instead of 400 for validation errors)
500 Internal Server Error	Unexpected server failure	A genuine bug/unhandled exception on the server side
503 Service Unavailable	Server temporarily down	Maintenance, overload, or a dependent service outage

4. 401 vs 403 — A Frequently Confused, Frequently-Tested Pair

401 Unauthorized: "I don't know WHO you are" (Authentication failure - Spring Security's Topic 1)
403 Forbidden: "I know who you are, but you're NOT ALLOWED to do this" (Authorization failure - Topic 2)

5. Real Life Analogy

- **Bank:** 401 is like a bank teller rejecting you because you HAVEN'T shown any valid ID at all. 403 is like the teller HAVING verified your ID successfully, but telling you "you're a REGULAR customer, and this specific vault room is reserved for VIP members only" — your identity is known and accepted, but this specific action still isn't permitted for you.

6. Edge Cases

Case	Result
Returning 200 OK for EVERYTHING, including creation/deletion/errors	A common anti-pattern — loses the self-descriptive nature of proper status codes, forcing clients to inspect the response BODY to figure out what actually happened
Returning 500 for a client's own bad input (should be 400)	Misleads monitoring/alerting systems into thinking there's a genuine SERVER bug, when it's actually a client mistake

7. Common Mistakes

- Returning 200 OK universally, regardless of the actual outcome — undermines the self-descriptive HTTP status code system entirely.
- Confusing 401 and 403 — a very common, frequently-tested interview mix-up.
- Using 500 for client-caused errors, muddying server error monitoring/alerting.

8. Frequently Asked Interview Questions

- 1 **What's the difference between 401 and 403?** 401 means the client isn't authenticated at all (identity unknown); 403 means the client IS authenticated, but lacks permission for the requested action.
- 2 **What status code should a successful POST that creates a resource return?** 201 Created.
- 3 **What status code should a successful DELETE typically return?** 204 No Content.
- 4 **Why is returning 200 OK for every possible outcome (including errors) a bad practice?** It defeats the self-descriptive purpose of HTTP status codes, forcing clients to parse the response body just to determine success/failure — status codes should communicate outcome at the protocol level itself.

TOPIC 4: REQUEST & RESPONSE

1. Introduction

(The Spring Boot-specific mechanics — @RequestBody, @PathVariable, @RequestParam, ResponseEntity — were covered in full with working code in the Spring Boot section's REST APIs topic. This topic focuses on the underlying HTTP MESSAGE STRUCTURE itself, independent of any specific framework.)

2. Anatomy of an HTTP Request

```
POST /api/employees HTTP/1.1 <- Request line: METHOD, PATH, HTTP VERSION
Host: api.company.com <- Headers
Content-Type: application/json
Authorization: Bearer eyJhbGc...
Accept: application/json
```

```
{"firstName": "Alice", "salary": 75000} <- Body (the actual data being sent)
```

3. Anatomy of an HTTP Response

```
HTTP/1.1 201 Created <- Status line: VERSION, STATUS CODE, REASON PHRASE
Content-Type: application/json
Location: /api/employees/42 <- often included on 201, pointing to the NEW resource
```

```
{"id": 42, "firstName": "Alice", "salary": 75000} <- Body
```

4. Key Headers to Know

Header	Purpose
Content-Type	What FORMAT the body is in (e.g., <code>application/json</code>)
Accept	What format the CLIENT wants the response in (content negotiation)
Authorization	Credentials/token (e.g., <code>Bearer <JWT></code> , from Spring Security)
Location	On 201 Created, the URI of the newly created resource
Cache-Control	Caching directives (ties to REST's Cacheability constraint, Topic 1)
ETag	A version identifier for a resource, useful for caching/conditional requests

5. Frequently Asked Interview Questions

- 1 **What three things make up an HTTP request's "request line"?** The HTTP method, the path/URI, and the HTTP version.
- 2 **What header commonly accompanies a 201 Created response, and what does it contain?** `Location` — the URI of the newly created resource.
- 3 **What's the difference between the `Content-Type` and `Accept` headers?** `Content-Type` describes the format of the BODY BEING SENT; `Accept` describes the format the CLIENT WANTS in the response (content negotiation).

TOPIC 5: DTOS (DATA TRANSFER OBJECTS)

1. Introduction

What is it? A DTO is a plain object designed SPECIFICALLY for transferring data across an API boundary — DELIBERATELY SEPARATE from your `@Entity` classes (Hibernate section), decoupling your PUBLIC API contract from your INTERNAL database schema.

2. Why NOT Return `@Entity` Objects Directly (Recap + Deeper Reasoning)

(The `LazyInitializationException` risk was covered in the Hibernate section. Here are the ADDITIONAL, API-design-specific reasons.)

Problem with returning <code>@Entity</code> directly	Why a DTO fixes it
Exposes INTERNAL database structure/column names as your PUBLIC API contract	A DTO defines its OWN shape, independent of however your database/entity is structured internally
Can accidentally expose SENSITIVE fields (password hashes, internal audit fields)	A DTO explicitly includes ONLY the fields meant to be public
Changing your database schema FORCES a breaking API change	A DTO acts as a STABLE contract layer, allowing internal schema changes without necessarily breaking API consumers
Risk of <code>LazyInitializationException</code> on unfetched relationships	DTOs are typically built WHILE the Hibernate Session is still open, or via a projection query that avoids the issue entirely

3. Code Example — Entity-to-DTO Mapping

```
// ENTITY (internal, database-mapped)
@Entity
public class Employee {
    @Id @GeneratedValue private Long id;
    private String firstName;
    private String lastName;
    private String passwordHash; // NEVER should be exposed publicly!
    private LocalDateTime lastLoginAt; // internal audit detail, not meant for the public API
    @ManyToOne private Department department;
}

// DTO (public, API-facing contract)
public record EmployeeResponseDto(
    Long id,
    String fullName, // COMBINES firstName+lastName - a DELIBERATE API design choice
    String departmentName // FLATTENS the nested Department relationship
) {
    public static EmployeeResponseDto fromEntity(Employee emp) {
        return new EmployeeResponseDto(
            emp.getId(),
            emp.getFirstName() + " " + emp.getLastName(),
            emp.getDepartment().getDeptName()
        );
    }
}

@GetMapping("/{id}")
public EmployeeResponseDto getEmployee(@PathVariable Long id) {
    Employee emp = employeeRepository.findById(id).orElseThrow();
    return EmployeeResponseDto.fromEntity(emp); // mapping happens WHILE the Session is still open - safe!
}
```

4. Mapping Libraries (For Larger Projects)

For projects with MANY entities/DTOs, hand-writing `fromEntity()/toEntity()` mapping methods becomes repetitive — libraries like **MapStruct** (compile-time code generation) or **ModelMapper** (reflection-based) automate this mapping, similar in spirit to how Spring Data JPA (previous section) eliminated repetitive DAO code.

5. Real Life Analogy

- **Bank:** An `@Entity` is like the bank's COMPLETE, raw internal customer record (including internal risk scores, audit trails, security notes). A DTO is like the CUSTOMER-FACING statement you actually receive — showing ONLY the relevant, appropriately-formatted information the customer is meant to see, deliberately hiding internal-only details.

6. Common Mistakes

- Returning `@Entity` objects directly from controllers "to save time," accidentally exposing sensitive/internal fields or hitting `LazyInitializationException`.
- Creating a SEPARATE DTO for every single controller method rather than reusing common ones — over-fragmentation without genuine benefit.

7. Frequently Asked Interview Questions

- 1 **What is a DTO, and why use one instead of returning entities directly?** A plain object designed for API data transfer, decoupling your public API contract from your internal database schema — avoiding exposure of sensitive fields, `LazyInitializationException` risk, and unnecessary API breakage from internal schema changes.
- 2 **Name a library that can automate entity-to-DTO mapping.** MapStruct (compile-time) or ModelMapper (reflection-based).
- 3 **Can a DTO combine/flatten fields differently from the underlying entity structure?** Yes — a DTO's shape is entirely independent, deliberately designed for what the API consumer actually needs (e.g., combining `firstName+lastName` into `fullName`).

TOPIC 6: VALIDATION (CROSS-REFERENCE)

(Fully covered with working code in the Spring Boot section, Topic 4 — `@Valid`, Bean Validation annotations (`@NotBlank`, `@Positive`, `@Email`, etc.), and the "forgot `@Valid`" gotcha apply identically to REST API design. No new content needed here — see that topic for full depth.)

TOPIC 7: VERSIONING

1. Introduction

What is it? API Versioning lets you evolve your API (change response formats, add/remove fields, alter behavior) over time WITHOUT breaking EXISTING clients still depending on the OLD behavior — critical for any API with external consumers who can't all update simultaneously.

2. The Four Common Versioning Strategies

Strategy 1: URI Path Versioning (Most Common, Simplest)

```
@RestController
@RequestMapping("/api/v1/employees")
public class EmployeeControllerV1 { }

@RestController
@RequestMapping("/api/v2/employees")
public class EmployeeControllerV2 { }
```

Strategy 2: Query Parameter Versioning

```
GET /api/employees?version=2
```

Strategy 3: Custom Header Versioning

```
GET /api/employees
X-API-Version: 2
```

Strategy 4: Media Type (Content Negotiation) Versioning

```
GET /api/employees
Accept: application/vnd.company.v2+json
```

3. Comparison Table

Strategy	Visibility	Caching-Friendly?	Common Usage
URI Path (/v1/, /v2/)	Very clear/explicit, visible in the URL itself	Yes (different URLs cache independently)	MOST common in practice — simple, clear
Query Parameter	Visible, but less "clean" URL structure	Yes	Less common
Custom Header	Clean URLs, version hidden in headers	Depends on cache configuration	Used by some enterprise APIs
Media Type	Most "purist"/RESTful (follows content negotiation principles)	Yes	Less common in practice, considered more complex

4. Real Life Analogy

- **Restaurant:** API versioning is like a restaurant chain maintaining BOTH an "OLD menu" and a "NEW menu" simultaneously for a transition period — existing regular customers who've memorized the old menu can keep ordering from it exactly as before, while new customers (or those ready to switch) can start using the updated menu, until the old one is eventually retired.

5. Edge Cases

Case	Result
A breaking change deployed WITHOUT any versioning strategy at all	IMMEDIATELY breaks every existing client depending on the old behavior — a serious, common real-world incident
Maintaining TOO MANY simultaneous versions indefinitely	Significant maintenance burden — a clear DEPRECATION and SUNSET policy for old versions is essential

6. Best Practices

- Choose URI path versioning (`/api/v1/`, `/api/v2/`) as the DEFAULT, simplest, most widely-understood approach unless there's a specific reason to prefer another strategy.
- Establish a clear DEPRECATION policy — announce, document, and eventually retire old versions on a defined timeline, rather than maintaining them forever.

7. Frequently Asked Interview Questions

- 1 **Why is API versioning necessary?** To allow the API to evolve (add/change/remove functionality) without breaking existing clients still depending on prior behavior.
- 2 **Name the four common versioning strategies.** URI path, query parameter, custom header, media type (content negotiation).
- 3 **Which versioning strategy is most common in practice, and why?** URI path versioning (`/api/v1/...`) — simplest, most explicit/visible, and naturally caching-friendly since different versions have distinct URLs.

TOPIC 8: ERROR HANDLING (CROSS-REFERENCE)

(Fully covered with working code in the Spring Boot section, Topic 8 — `@RestControllerAdvice` + `@ExceptionHandler` for centralized, consistent error responses. Combine this with Topic 3's proper status code selection for a complete REST error-handling strategy. No new content needed here — see that topic for full depth.)

TOPIC 9: HATEOAS

1. Introduction

What is it? HATEOAS (Hypermedia As The Engine Of Application State) is the REST constraint (mentioned in Topic 1's Uniform Interface) where API responses include LINKS to related actions/resources — letting a client DISCOVER what it can do NEXT dynamically from the response itself, rather than needing that knowledge HARDCODED in advance.

2. Without HATEOAS (Typical, Most Common Real-World APIs)

```
{
  "id": 42,
  "firstName": "Alice",
  "status": "PENDING_APPROVAL"
}
```

The CLIENT must already KNOW (from separate documentation) that a "PENDING_APPROVAL" order can be approved via a hardcoded POST `/api/orders/42/approve` endpoint — this knowledge lives ENTIRELY in the client's own code/documentation, not in the API response itself.

3. With HATEOAS

```
{
  "id": 42,
  "firstName": "Alice",
  "status": "PENDING_APPROVAL",
  "_links": {
    "self": { "href": "/api/orders/42" },
    "approve": { "href": "/api/orders/42/approve", "method": "POST" },
    "cancel": { "href": "/api/orders/42/cancel", "method": "POST" }
  }
}
```

The client can now DISCOVER available NEXT actions dynamically from the response ITSELF — if the order were ALREADY approved, the response simply wouldn't include an "approve" link at all, and the client would know NOT to offer that action, WITHOUT needing separately-maintained business-rule logic duplicated on the client side.

4. Spring HATEOAS — Code Example

```
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-hateoas</artifactId>
</dependency>

import org.springframework.hateoas.EntityModel;
import static org.springframework.hateoas.server.mvc.WebMvcLinkBuilder.*;

@GetMapping("/{id}")
public EntityModel<Employee> getEmployee(@PathVariable Long id) {
    Employee emp = employeeRepository.findById(id).orElseThrow();

    return EntityModel.of(emp,
        linkTo(methodOn(EmployeeController.class).getEmployee(id)).withSelfRel(),
        linkTo(methodOn(EmployeeController.class).getAllEmployees()).withRel("all-employees"));
}
```

5. Real Life Analogy

- **Restaurant:** HATEOAS is like a physical menu that, alongside each dish, ALSO prints exactly what you can do NEXT ("Order dessert," "Request the bill") — directly ON the menu itself, discoverable in the moment, rather than requiring you to have separately memorized (from an entirely different, unrelated pamphlet) the full list of every possible follow-up action available at a restaurant.

6. Why Most Real-World APIs SKIP HATEOAS

Despite being a core REST principle, HATEOAS is **genuinely rare in practice** — most real APIs (including the ones you'll build day-to-day) are simpler, plain JSON APIs without hypermedia links, because:

- It adds real implementation complexity for MODEST practical benefit in typical use cases.
- Most API consumers (especially internal, tightly-coupled frontend teams) already KNOW the API's capabilities from documentation, making dynamic discovery less valuable.
- It's most valuable for PUBLIC, LONG-LIVED, loosely-coupled APIs consumed by MANY independent, evolving clients — a genuinely less common scenario than typical internal service-to-service or frontend-to-backend APIs.

7. Frequently Asked Interview Questions

- 1 **What does HATEOAS stand for?** Hypermedia As The Engine Of Application State.
- 2 **What problem does HATEOAS solve?** Letting clients discover available next actions/related resources DYNAMICALLY from the API response itself, rather than requiring that knowledge to be separately hardcoded in client code/documentation.
- 3 **Why do most real-world "RESTful" APIs skip HATEOAS despite it being a core REST principle?** It adds real implementation complexity for often-modest practical benefit, especially for internal/tightly-coupled APIs where clients already know the API's capabilities from documentation — it's most valuable for public, loosely-coupled APIs with many independent evolving consumers.

TOPIC 10: OPENAPI/SWAGGER (CROSS-REFERENCE)

(Fully covered with working setup steps in the Spring Boot section, Topic 13 —

springdoc-openapi-starter-webmvc-ui, auto-generated interactive documentation at /swagger-ui.html, and @Operation/@ApiResponse/@Tag annotations. No new content needed here — see that topic for full depth.)

Revision Notes (Entire REST APIs Section Consolidated)

REST APIS – CHEAT SHEET

=====

REST = an ARCHITECTURAL STYLE (Roy Fielding, 2000), NOT a protocol
6 CONSTRAINTS: Client-Server, Stateless, Cacheable, Uniform Interface, Layered System, Code on Demand (optional)
Most "RESTful" APIs in practice DON'T fully implement HATEOAS - a real, common nuance

HTTP METHODS: GET(safe,idempotent) POST(neither) PUT(idempotent,REPLACES all fields)
PATCH(partial update) DELETE(idempotent)
PUT vs PATCH: PUT can silently NULL missing fields - a common real bug

STATUS CODES: 200 OK, 201 Created (+Location header), 204 No Content, 400 Bad Request, 401 Unauthorized (WHO), 403 Forbidden (WHAT), 404 Not Found, 409 Conflict, 500 Internal Server Error

REQUEST/RESPONSE: method+path+headers+body ; status+headers+body
Content-Type (what I'm sending) vs Accept (what I want back)

DTOs: NEVER return @Entity directly - hides sensitive fields, decouples API from DB schema, avoids LazyInitializationException
(Validation, Error Handling, Swagger -> see Spring Boot section)

VERSIONING: URI path (/api/v1/) is the simplest/most common; also query param, custom header, media type strategies
HATEOAS: responses include discoverable _links to next actions - rare in practice despite being core REST, most valuable for public/loosely-coupled APIs

Homework (Covers Entire REST APIs Section)

- 1 Design a small API (e.g., "Order" resource) choosing the correct HTTP method and status code for each of: list all, get one, create, full update, partial update, delete.
- 2 Deliberately implement an endpoint using `PUT` that only updates SOME fields (missing ones from the request), and observe fields being unintentionally nulled — then fix it by switching to `PATCH` semantics.
- 3 Add URI-path versioning (`/api/v1/` and `/api/v2/`) to a small existing controller, with `v2` changing one field's shape.
- 4 Add basic HATEOAS links (self + one related action) to a single endpoint using Spring HATEOAS.

FINAL SUMMARY — REST APIS COMPLETE

This concludes the **REST APIs** section — 10 topics:

- 1 **REST Principles** — Fielding's six constraints, "RESTful" vs strictly REST-compliant.
- 2 **HTTP Methods** — safe vs idempotent, the `PUT` vs `PATCH` field-nulling trap.
- 3 **Status Codes** — full reference, the `401` vs `403` distinction.
- 4 **Request & Response** — raw HTTP message anatomy, key headers.
- 5 **DTOs** — why never return entities directly, mapping strategies.
- 6 **Validation** — cross-referenced to Spring Boot.

- 7 **Versioning** — the four strategies, URI path as the practical default.
- 8 **Error Handling** — cross-referenced to Spring Boot.
- 9 **HATEOAS** — the theory, working Spring HATEOAS code, and why it's rare in practice.
- 10 **OpenAPI/Swagger** — cross-referenced to Spring Boot.

This section formalizes the architectural principles behind everything you've built across Spring Boot and Spring Security — you now understand not just HOW to call `@GetMapping`, but WHY REST is structured the way it is. Next: Maven, covering the build tool underlying every project you've built so far.

(END OF REST APIS MASTER NOTES — ready to proceed to Maven whenever you say "Next Topic.")